q3

April 3, 2025

1 Assignment 2 - Question 3: Tree-based Models

Course Name: Machine Learning (DDA3020)

(30 points in total)

Please enter your personal information (Double-click this block first)

Name: Chen Ziqi

Student ID: 123090059

1.0.1 Overview

In this question, you will use three tree-based models (Decision Tree, Bagging, and Random Forest) to solve a real problem, image classification. The tasks includes training models by different parameters, comparing the results of different models, and trying to find the best parameter combination. As a part of an assignment, your task is to **run all codes in this script and complete the parts marked with** [TASK].

1.0.2 Dataset Introduction

Since we will use a same dataset for both question 3 & 4, it's important to have an overall knowledge of this dataset. The dataset is called **Fashion-MNIST**, a member of the MNIST family, but I think it's more interesting. This dataset is an image classification task with ten categories, all of which are items from our daily lives. It consists of a training set of 60,000 examples and a test set of 10,000 examples. Each example is a 28x28 grayscale image. It means that we can use 784 numbers (in the range of 0~255) to represent one image. More details can be found in https://github.com/zalandoresearch/fashion-mnist.

OK. Now it's time to start, please read and run each block of codes in order, so that you will have a smooth experience.

[3]:
```
import numpy as np
import gzip
import random
import os
import pandas as pd
import matplotlib.pyplot as plt

# Please do not modify the seed number here.
```

```
np.random.seed(336)
```

Don't need to carefully read this block since it's just loading the dataset. Just run it.

[4]:
```
def load_mnist(path, kind, subset=None):
    labels_path = os.path.join(path, '%s-labels-idx1-ubyte.gz'%kind)
    images_path = os.path.join(path, '%s-images-idx3-ubyte.gz'%kind)
    with gzip.open(labels_path, 'rb') as lbpath:
        labels = np.frombuffer(lbpath.read(), dtype=np.uint8, offset=8)
    with gzip.open(images_path, 'rb') as imgpath:
        images = np.frombuffer(imgpath.read(), dtype=np.uint8, offset=16).
 ↪reshape(len(labels), 784)

    if subset is not None:
        selected_images, selected_labels = [], []
        for label in range(10):
            indices = np.where(labels == label)[0]
            selected_indices = np.random.choice(indices, subset, replace=False)
            selected_images.append(images[selected_indices])
            selected_labels.append(labels[selected_indices])
        images = np.concatenate(selected_images, axis=0)
        labels = np.concatenate(selected_labels, axis=0)

        paired = list(zip(images, labels))
        random.shuffle(paired)
        images, labels = zip(*paired)

    return np.array(images), np.array(labels)
```

In this question, we only use a subset of Fashion-MNIST dataset to reduce compuration time.

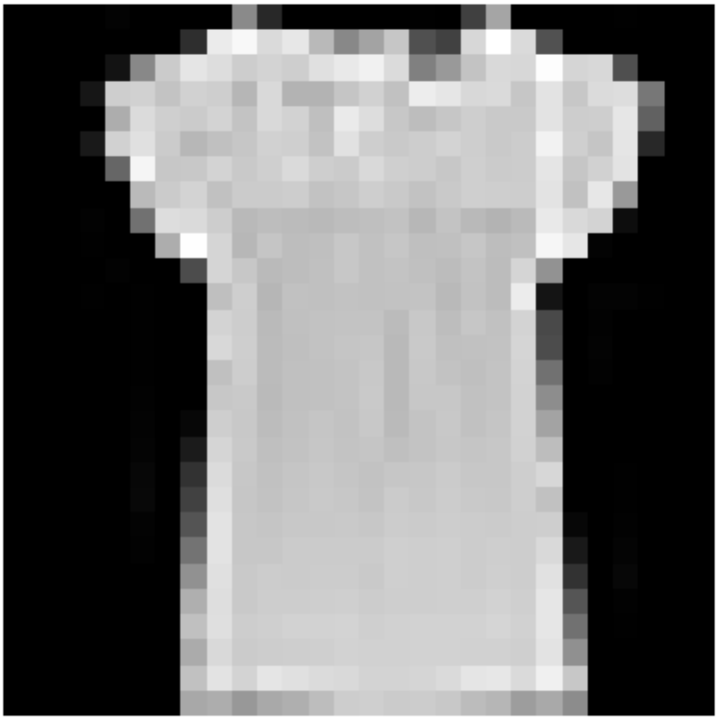
[5]:
```
X_train, y_train = load_mnist('./data/', kind='train', subset=100)
X_test, y_test = load_mnist('./data/', kind='t10k', subset=20)
```

You can repeat running this block for several times to see different images with its label in this dataset.

[6]:
```
label_names = ["T-shirt/
 ↪top","Trouser","Pullover","Dress","Coat","Sandal","Shirt","Sneaker","Bag","Ankle␣
 ↪boot"]
idx = random.choice(range(0,len(X_test)))
print(f"The label index of this image is: {y_test[idx]}")
print(f"The label name of this graph is: {label_names[y_test[idx]]}")

plt.imshow(X_test[idx].reshape(28, 28), cmap='gray')
plt.axis('off')
plt.show()
```

```
The label index of this image is: 0
The label name of this graph is: T-shirt/top
```



In this question, we use each pixel to represent different features of a data sample. It means that each sample will have 784 features to learn and 1 label to predict. The advantage is that you don't need to struggle on data processing but focus on training models and analyzing results.

[7]:
```
X_train = pd.DataFrame(X_train, columns=[f"pixel_{i}" for i in range(X_train.
 ↪shape[1])])
X_test = pd.DataFrame(X_test, columns=[f"pixel_{i}" for i in range(X_test.
 ↪shape[1])])
```

In both question 3 & 4, we only use one metric, **accuracy_score**. Here is a simple example about how to use this function from sklearn.

[8]:
```
from sklearn.metrics import accuracy_score
_y_true = [0,1,1,2,0]
_y_pred = [1,1,2,2,0]
accuracy_score(_y_true, _y_pred)
```

[8]:
```
0.6
```

1.0.3 Task 1

Write code to train Decision Tree Classification models with different **max_depth** (=1,2,...,9). Save the accuracy scores of **both** predicting train data and test data respectively in train_acc_dt and test_acc_dt. (For further analysis) [TASK] (5 points)

[9]:
```
from sklearn.tree import DecisionTreeClassifier
train_acc_dt = []
test_acc_dt  = []
depth_list = range(1, 10)

##########################
## Write your code here ##
##########################

for depth in depth_list:
    dt_classifier = DecisionTreeClassifier(max_depth=depth, random_state=336)

    dt_classifier.fit(X_train, y_train)

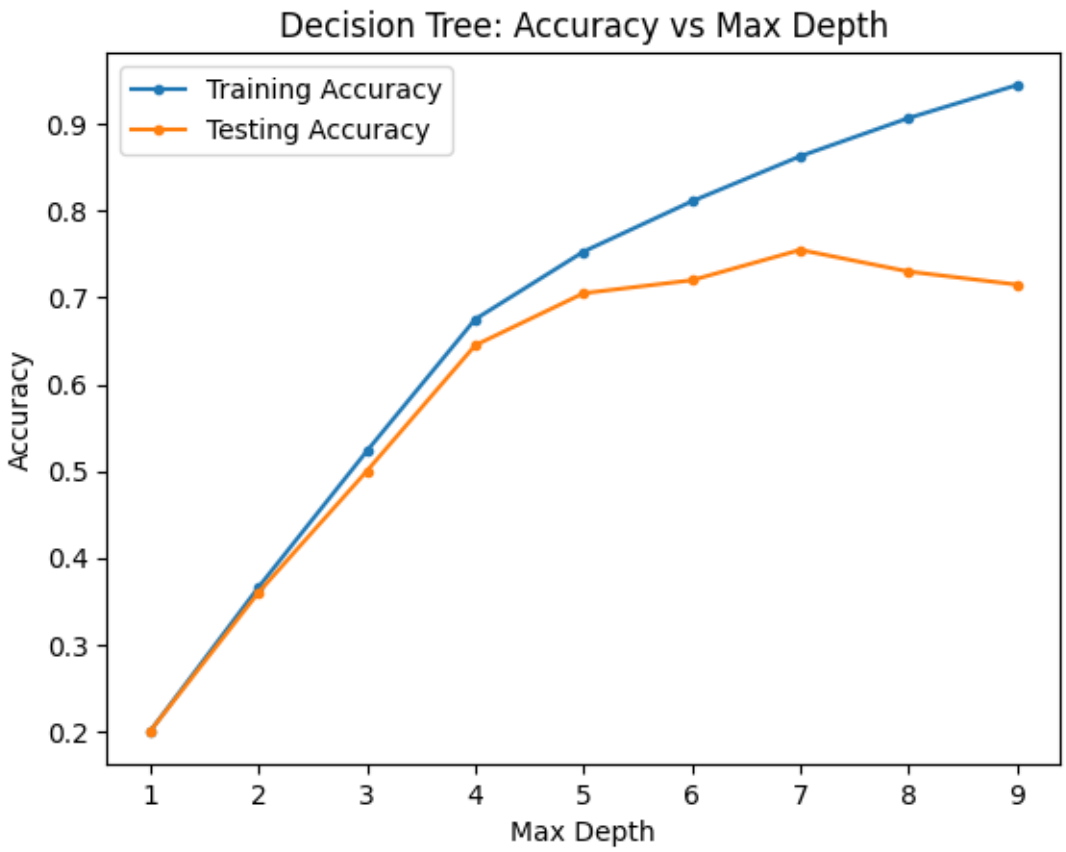
    y_train_pred = dt_classifier.predict(X_train)
    train_accuracy = accuracy_score(y_train, y_train_pred)
    train_acc_dt.append(train_accuracy)

    y_test_pred = dt_classifier.predict(X_test)
    test_accuracy = accuracy_score(y_test, y_test_pred)
    test_acc_dt.append(test_accuracy)

    print(f"Depth {depth}: Train Acc = {train_accuracy:.4f}, Test Acc =␣
 ↪{test_accuracy:.4f}")

plt.plot(depth_list, train_acc_dt, marker=".", label='Training Accuracy')
plt.plot(depth_list, test_acc_dt, marker=".", label='Testing Accuracy')
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.title('Decision Tree: Accuracy vs Max Depth')
plt.legend()
plt.show()
```

```
Depth 1: Train Acc = 0.2000, Test Acc = 0.2000
Depth 2: Train Acc = 0.3660, Test Acc = 0.3600
Depth 3: Train Acc = 0.5230, Test Acc = 0.5000
Depth 4: Train Acc = 0.6750, Test Acc = 0.6450
Depth 5: Train Acc = 0.7530, Test Acc = 0.7050
Depth 6: Train Acc = 0.8110, Test Acc = 0.7200
Depth 7: Train Acc = 0.8630, Test Acc = 0.7550
Depth 8: Train Acc = 0.9070, Test Acc = 0.7300
Depth 9: Train Acc = 0.9450, Test Acc = 0.7150
```



Here you will have a graph about accuracy scores of both train data and test data. **Please write your observations and findings here.** [TASK] (3 points)

Your answer: - When `Max Depth` is less than or equal to **7**, the testing accuracy increases so as the training accuracy. - However, when `Max Depth` exceeds **7**, the testing accuracy decreases while the training accuracy keeps increasing. - So after `Max Depth` exceeds **7**, the model is likely to **overfit** for the training data, which leads to the decreasing accuracy for the testing data.

1.0.4 Task 2

Write code to train Random Forest Classification models with different **max_depth** (=1,2,...,9). Save the accuracy scores of **both** predicting train data and test data respectively in train_acc_rf and test_acc_rf. (For further analysis) [TASK] (5 points)

[10]:
```
from sklearn.ensemble import RandomForestClassifier
train_acc_rf = []
test_acc_rf  = []
depth_list = range(1, 10)

#
for depth in depth_list:
```

```
    #       max_depth
    rf_classifier = RandomForestClassifier(max_depth=depth, random_state=336)

    #
    rf_classifier.fit(X_train, y_train)

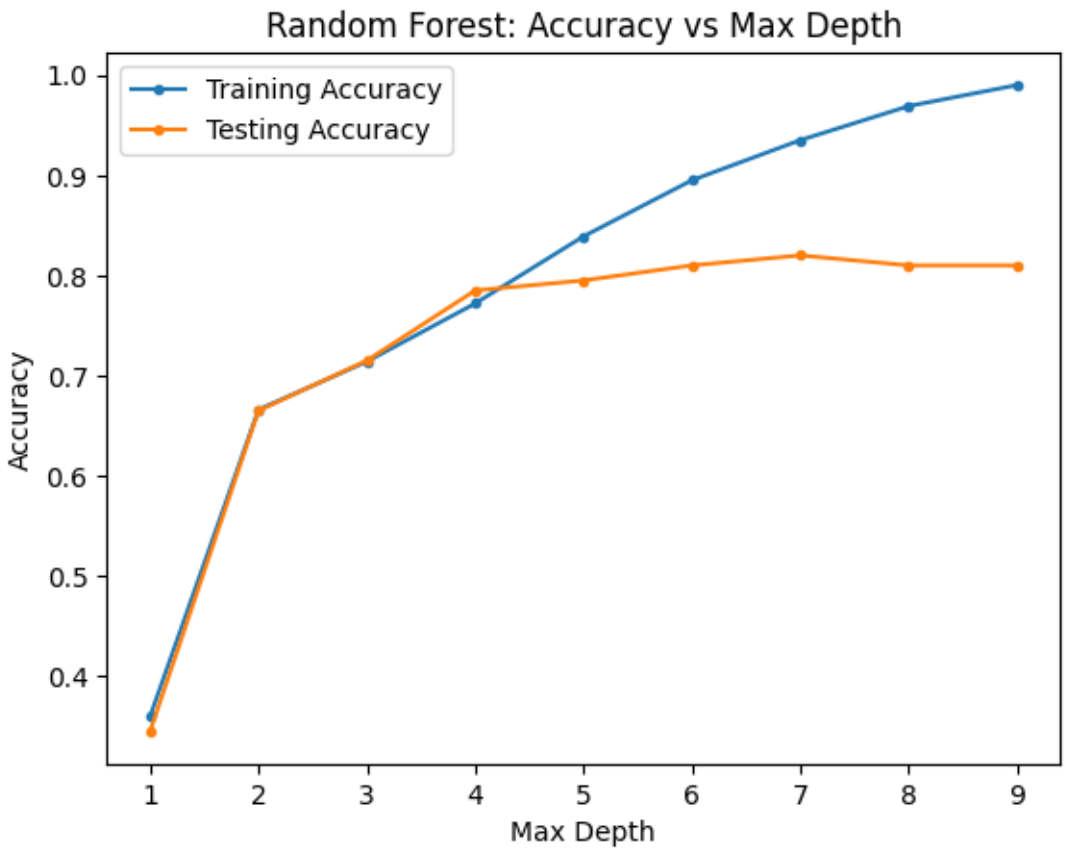
    #
    y_train_pred = rf_classifier.predict(X_train)
    train_accuracy = accuracy_score(y_train, y_train_pred)
    train_acc_rf.append(train_accuracy)

    #
    y_test_pred = rf_classifier.predict(X_test)
    test_accuracy = accuracy_score(y_test, y_test_pred)
    test_acc_rf.append(test_accuracy)

    #
    print(f"Depth {depth}: Train Acc = {train_accuracy:.4f}, Test Acc =␣
 ↪{test_accuracy:.4f}")

plt.plot(depth_list, train_acc_rf, marker=".", label='Training Accuracy')
plt.plot(depth_list, test_acc_rf, marker=".", label='Testing Accuracy')
plt.xlabel('Max Depth')
plt.ylabel('Accuracy')
plt.title('Random Forest: Accuracy vs Max Depth')
plt.legend()
plt.show()
```

```
Depth 1: Train Acc = 0.3600, Test Acc = 0.3450
Depth 2: Train Acc = 0.6660, Test Acc = 0.6650
Depth 3: Train Acc = 0.7140, Test Acc = 0.7150
Depth 4: Train Acc = 0.7720, Test Acc = 0.7850
Depth 5: Train Acc = 0.8390, Test Acc = 0.7950
Depth 6: Train Acc = 0.8950, Test Acc = 0.8100
Depth 7: Train Acc = 0.9350, Test Acc = 0.8200
Depth 8: Train Acc = 0.9690, Test Acc = 0.8100
Depth 9: Train Acc = 0.9900, Test Acc = 0.8100
```



Here you will have a graph about accuracy scores of both train data and test data. **Try to compare this graph with the above one (Decision Tree's graph), and write your observations and findings here.** [TASK] (3 points)

Your answer: - As `Max Depth` increases, the testing accuracy increases as well as the training accuracy. - Compared with the above one (Decision Tree), the testing accuracy for Random Forest decreases at larger `Max Depth`, indicating that Random Forest can decrease the effect of overlap.

1.0.5 Task 3

Write code to train **both Bagging and Random Forest** Classification models with different **n_estimators** (=10,20,50,100). Save the accuracy scores of predicting test data for both models respectively in bagging_test_acc and rf_test_acc. (For further analysis) [TASK] (5 points)

[13]:
```
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import RandomForestClassifier
n_estimators_list = [10, 20, 50, 100]
bagging_test_acc = []
rf_test_acc = []

##########################
```

```
## Write your code here ##
##########################

for n_estimators in n_estimators_list:

    bagging_classifier = BaggingClassifier(n_estimators=n_estimators,␣
 ↪random_state=336)
    bagging_classifier.fit(X_train, y_train)

    bagging_y_pred = bagging_classifier.predict(X_test)
    bagging_accuracy = accuracy_score(y_test, bagging_y_pred)
    bagging_test_acc.append(bagging_accuracy)

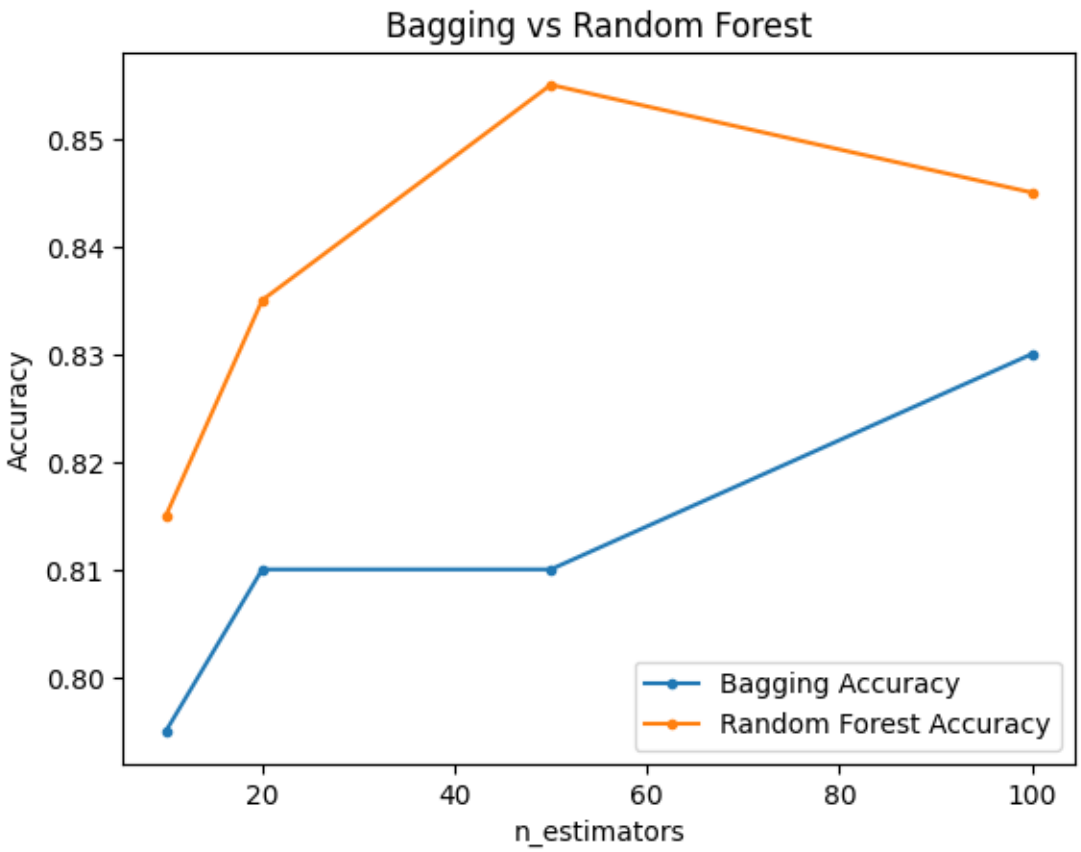
    rf_classifier = RandomForestClassifier(n_estimators=n_estimators,␣
 ↪random_state=336)
    rf_classifier.fit(X_train, y_train)

    rf_y_pred = rf_classifier.predict(X_test)
    rf_accuracy = accuracy_score(y_test, rf_y_pred)
    rf_test_acc.append(rf_accuracy)

    print(f"n_estimators={n_estimators}: Bagging Acc={bagging_accuracy:.4f}, RF␣
 ↪Acc={rf_accuracy:.4f}")

plt.plot(n_estimators_list, bagging_test_acc, marker=".", label='Bagging␣
 ↪Accuracy')
plt.plot(n_estimators_list, rf_test_acc, marker=".", label='Random Forest␣
 ↪Accuracy')
plt.xlabel('n_estimators')
plt.ylabel('Accuracy')
plt.title('Bagging vs Random Forest')
plt.legend()
plt.show()
```

```
n_estimators=10: Bagging Acc=0.7950, RF Acc=0.8150
n_estimators=20: Bagging Acc=0.8100, RF Acc=0.8350
n_estimators=50: Bagging Acc=0.8100, RF Acc=0.8550
n_estimators=100: Bagging Acc=0.8300, RF Acc=0.8450
```



Here you will have a graph about accuracy scores of both models. **Please write your observations and findings here.** [TASK] (3 points)

Your answer: - In general, the accuracy of Random Forest is higher than Bagging Accuracy at the same `n_estimators`. - However, the accuracy of Random Forest decreases when `n_estimators` grows larger, while Bagging accuracy keeps increasing.

1.0.6 Task 4

Write code to find the best parameter combination from **[n_estimators=50/100/200, max_depth=5/10/20/50]** by 4-fold cross validation. Please use GridSearchCV from sklearn to implement that. Output the best parameter combination and the test accuracy score with the best parameters. [TASK] (6 points)

[12]:
```
from sklearn.model_selection import GridSearchCV

############################
## Write your code here ##
############################

param_grid = {
```

```
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 20, 50]
}

rf_classifier = RandomForestClassifier(random_state=336)

grid_search = GridSearchCV(
    estimator=rf_classifier,
    param_grid=param_grid,
    cv=4,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

best_params = grid_search.best_params_

best_rf = RandomForestClassifier(**best_params, random_state=336)
best_rf.fit(X_train, y_train)

y_test_pred = best_rf.predict(X_test)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("Best params:", best_params)
print("Test Accuracy with Best Model: ", test_accuracy)
```

```
Best params: {'max_depth': 20, 'n_estimators': 200}
Test Accuracy with Best Model:  0.85
```